Introdução ao Spring Security e JWT

Material introdutório para o 3º ano do Ensino Médio Integrado

Objetivos da aula

Ao final desta leitura, você deverá conseguir explicar:

- por que uma API precisa de segurança;
- a diferença entre autenticação e autorização;
- o papel do Spring Security;
- o que é um JWT e como ele participa de uma requisição;
- cuidados básicos com senhas, tokens e permissões.

> Este material apresenta os conceitos antes da implementação. A API ImageLite ainda precisa de configuração específica para autenticação e autorização.

1. Por que proteger uma API?

Uma API permite que programas troquem informações. No ImageLite, por exemplo, a interface web consulta a API para listar imagens e pode enviar uma nova imagem.

Se todas as operações forem públicas, qualquer pessoa que consiga acessar a API poderá tentar executá-las. Uma API protegida pode exigir que o usuário se identifique e pode limitar o que cada pessoa está autorizada a fazer.

Pense no acesso a uma escola:

- **Identificação:** você informa quem diz ser.
- **Autenticação:** a escola confere se a informação é verdadeira.
- **Autorização:** depois de confirmar sua identidade, a escola verifica a

quais salas ou recursos você pode ter acesso.

Na aplicação, uma rota pode ser pública, exigir que o usuário esteja autenticado ou exigir uma permissão específica.

2. Autenticação e autorização

Os conceitos são relacionados, mas respondem a perguntas diferentes:

- **Autenticação:** “Quem é você?” Por exemplo, conferir usuário e senha.
- **Autorização:** “O que você pode fazer?” Por exemplo, permitir que apenas

um administrador remova imagens.

Uma pessoa pode estar autenticada e ainda não ter autorização para determinada operação. Esconder um botão na interface não substitui a verificação de permissão no servidor: alguém poderia tentar chamar a API diretamente.

3. O que é Spring Security?

Spring Security é o conjunto de ferramentas do ecossistema Spring usado para proteger aplicações. Ele ajuda a exigir autenticação, controlar permissões e definir quais rotas são públicas.

De forma simplificada, as requisições passam por uma cadeia de filtros antes de chegar ao controller:

```
Requisição HTTP
      |
      v
Cadeia de filtros do Spring Security
      |
      +--> credenciais ausentes ou inválidas: acesso recusado
      |
      +--> credenciais válidas: requisição continua
                                |
                                v
                        Controller da API
```

Uma configuração moderna do Spring Security costuma declarar as regras em um bean chamado `SecurityFilterChain`. Esse bean descreve quais rotas podem ser acessadas publicamente e quais exigem autenticação ou permissões.

4. O que é JWT?

JWT é a sigla de **JSON Web Token**. É um formato padronizado para representar informações que podem ser transmitidas entre sistemas.

Um JWT costuma ter três partes, separadas por pontos:

```
cabecalho.conteudo.assinatura
```

1. **Header (cabeçalho):** informa o tipo do token e o algoritmo de assinatura.
2. **Payload (conteúdo):** contém claims, isto é, informações como o usuário,

o emissor e a data de expiração.

1. **Signature (assinatura):** permite verificar se o token foi assinado por

quem possui a chave correspondente e se as partes assinadas foram alteradas.

Um token assinado não é necessariamente secreto

O cabeçalho e o payload de um JWT assinado normalmente são codificados, não criptografados. Quem obtiver o token pode decodificar e ler essas partes. Por isso, **não coloque senhas, documentos ou outros dados secretos no payload**.

A assinatura ajuda a verificar a integridade e a origem do token; ela não esconde seu conteúdo. Um JWT assinado não é a mesma coisa que uma mensagem criptografada.

5. Como funciona um login com JWT?

Um fluxo comum em uma API é:

1. O usuário envia suas credenciais para uma rota de login por uma conexão

HTTPS.

1. O servidor procura a conta e compara a senha informada com o hash guardado.
2. Se a autenticação for aprovada, o servidor cria e assina um token com as

informações necessárias e uma data de expiração.

1. O servidor devolve o token ao cliente.
2. Para acessar uma rota protegida, o cliente envia o token no cabeçalho HTTP:

```
Authorization: Bearer <token>
```

1. O servidor verifica a assinatura, a validade e as demais regras do token.
2. Se o token for válido e o usuário tiver permissão, a requisição continua.

Caso contrário, o servidor recusa o acesso.

O servidor não deve aceitar um token apenas porque ele "parece" um JWT: a assinatura e as regras de validade precisam ser verificadas.

6. Como as peças do Spring Security se relacionam?

- `SecurityFilterChain`: define as regras de segurança e quais requisições

podem prosseguir.

- `AuthenticationManager`: coordena uma tentativa de autenticação.
- `UserDetailsService`: busca os dados de usuário usados no processo, de

acordo com a aplicação.

- `PasswordEncoder`**:** compara uma senha informada com o hash armazenado.
- **Filtro de autenticação:** em uma configuração JWT, pode ler o cabeçalho

`Authorization`, validar o token e informar ao Spring quem fez a requisição.

- **Authorities e roles:** representam permissões associadas ao usuário e

podem ser usadas nas regras de autorização.

Esses componentes precisam ser configurados de acordo com a arquitetura. Nem toda aplicação precisa escrever seu próprio filtro JWT: o Spring Security também pode atuar como servidor de recursos OAuth 2.0 e validar tokens emitidos por um provedor compatível.

7. JWT e sessão

Em uma autenticação por sessão, o servidor mantém informações da sessão e o cliente costuma receber um cookie com um identificador. Nas requisições seguintes, o servidor usa esse identificador para localizar a sessão.

Com JWT, o cliente envia o token em cada requisição e o servidor valida o token recebido. Em muitas APIs, esse modelo permite não manter uma sessão individual para cada cliente.

JWT não é sempre a melhor escolha. A decisão depende dos requisitos:

- Sessões podem ser encerradas diretamente no servidor.
- Um JWT emitido pode continuar válido até expirar; revogá-lo antes disso

exige planejamento.

- Nos dois modelos, é necessário proteger as credenciais e definir regras

adequadas de autenticação e autorização.

8. Cuidados de segurança

1. **Nunca guarde senhas em texto puro.** Armazene hashes usando uma função

própria para senhas, configurada adequadamente, como BCrypt ou Argon2. Base64 e criptografia reversível não substituem hash de senha.

1. **Use HTTPS fora do ambiente local de desenvolvimento.** HTTPS protege os

dados durante o transporte entre cliente e servidor.

1. **Não coloque segredos no payload do JWT.** O conteúdo de tokens assinados

pode ser lido por quem possui o token.

1. **Proteja a chave de assinatura.** Não a escreva diretamente no código nem

a envie para o repositório.

1. **Valide o token.** Confira assinatura, algoritmo permitido, emissor e

expiração. Não confie nos dados apenas porque aparecem no payload.

1. **Defina um prazo de validade.** Um token roubado pode ser usado até deixar

de ser aceito.

1. **Planeje a revogação.** Considere o que fazer em caso de logout, troca de

senha ou roubo do token.

1. **Não exponha tokens.** Evite colocá-los em URLs, logs, capturas de tela ou

código compartilhado.

1. **Confira permissões no servidor.** Ocultar um botão na interface não

bloqueia uma chamada direta à API.

9. Códigos HTTP relacionados

- **200 OK:** a requisição foi processada.
- **201 Created:** um recurso foi criado.
- **401 Unauthorized:** não foi apresentada uma autenticação válida.
- **403 Forbidden:** o usuário foi autenticado, mas não tem permissão para

realizar a operação.

10. Aplicando os conceitos ao ImageLite

Hoje, a galeria consulta `GET /images` e o formulário pode enviar imagens para `POST /images`. Ao planejar a segurança do ImageLite, a turma poderia discutir:

- A galeria deve ser pública?
- Só usuários autenticados podem enviar imagens?
- Cada pessoa pode alterar somente as imagens que enviou?
- Deve existir uma permissão especial para operações administrativas?

Essas são decisões de projeto. Adicionar JWT sem definir quais rotas e ações precisam de proteção não torna a aplicação automaticamente segura.

Glossário

- **API:** interface que permite que aplicações troquem informações.
- **Bearer token:** credencial que dá acesso a quem a apresentar; deve ser

mantida em segredo.

- **Claim:** informação declarada no payload de um token.
- **Hash:** resultado de uma função de mão única, usado, por exemplo, para

guardar senhas sem armazenar o valor original.

- **HTTPS:** HTTP protegido por criptografia durante o transporte.
- **JWT:** formato de token com cabeçalho, conteúdo e assinatura.
- **Token:** dado apresentado por um cliente para provar identidade ou

permissão, de acordo com as regras do sistema.

Para conversar em sala

1. Qual é a diferença entre autenticar uma pessoa e autorizá-la?
2. Por que o payload de um JWT não deve conter a senha?
3. O que pode acontecer se alguém copiar um token ainda válido?
4. Por que esconder um botão não substitui uma regra de autorização na API?
5. No ImageLite, quais operações poderiam ser públicas e quais deveriam exigir

autenticação? Justifique.

Próximo passo

Antes de implementar autenticação no ImageLite, definam em grupo:

1. quais rotas serão públicas;
2. quais rotas exigirão login;
3. quais papéis ou permissões existirão;
4. como as senhas serão armazenadas;
5. como serão protegidos, expirados e revogados os tokens.

Depois, consultem a documentação oficial do Spring Security para a versão usada pelo projeto. Este backend usa Spring Boot 4.0.3 e Java 21; dependências e exemplos de configuração devem ser conferidos para essas versões.